

Όνοματεπώνυμο : Μάριος Ελληνίδης

AM:4926

Link: <https://github.com/uoibrunch/CNNSearchEngine>

News Article Search Engine

Περιγραφή

Στο project στοχεύω στην υλοποίηση ενός συστήματος αναζήτησης πληροφορίας από ειδησεογραφικά άρθρα του cnn με διάφορα πεδία όπως συγγραφέας , ημερομηνία δημοσίευσης , κατηγορία , τμήμα , επικεφαλίδα , περιγραφή και keywords. Στόχος είναι να δημιουργήσω ένα σύστημα το οποίο θα δημιουργεί ένα αποτελεσματικό ευρετήριο για αυτά τα άρθρα και θα επιτρέπει την αναζήτησή και ανάκτηση με βάση την συνάφεια.

Συλλογή εγγράφων

Τα δεδομένα του project προέρχονται από μια συλλογή δεδομένων από άρθρα ειδήσεων του cnn . Κάθε άρθρο στην συλλογή περιέχει τα παρακάτω πεδία:

1. **Author:** Ο συγγραφέας του άρθρου , αυτο τα πεδίο μπορεί να είναι χρήσιμα για αναζητήσεις που επιγκετρώνονται σε άρθρα από συγκεκριμένο συγγραφέα
2. **Category:** Η κατηγορία του άρθρου(π.χ. Πολιτική , Οικονομία , Αθλητικά). , αυτό το πεδίο επιγκεντρώνεται σε άρθρα σε συγκεκριμένη κατηγορία , ωστόσο όπως και ο συγγραφέας φαίνεται να είναι λιγότερο σημαντικά
3. **Section:** Η ενότητα στην οποία ανήκει το άρθρο (π.χ. Διεθνή, Εθνικά), φαίνεται λιγότερο σημαντικό.
4. **Headline:** Ο τίτλος του άρθρου , αυτό το πεδίο είναι σημαντικό για την αναζήτησή , καθώς οι χρήστες συχνά θα αναζητούν άρθρα με βάση τον τίτλο του. Η επικεφαλίδα μπορεί να περιέχει λέξεις κλειδιά που καθορίζουν το περιεχόμενο του άρθρου.
5. **Description:** Μια σύντομη περιγραφή του περιεχομένου του άρθρου. Η περιγραφή είναι χρήσιμη για την αναζήτησή , καθώς συνήθως περιγράφει το περιεχόμενο του άρθρου και μπορεί να περιλαμβάνει σημαντικές λέξεις κλειδιά.
6. **Article Text:** Το πλήρες κείμενο του άρθρου. Αυτό είναι το κύριο κείμενο του άρθρου και φυσικά θα χρησιμοποιηθεί για την αναζήτησή. Το κείμενο περιλαμβάνει όλες τις λέξεις του άρθρου και αποτελεί την πιο σημαντική πηγή για αναζητήσεις βάσει περιεχομένου.

7. **Headline:** Ο τίτλος του άρθρου. Αυτό το πεδίο είναι σημαντικό για αναζήτησή , καθώς οι χρήστες συχνά θα αναζητούν άρθρα με βάση τον τίτλο τους. Η επικεφαλίδα μπορεί να περιέχει λέξεις κλειδιά που καθορίζουν το περιεχόμενο του άρθρου.
8. **Description:** Μια σύντομη περιγραφή του περιεχομένου του άρθρου. Η περιγραφή είναι χρήσιμη για την αναζήτησή , καθώς συνήθως περιγράφει το περιεχόμενο του άρθρου και μπορεί να περιλαμβάνει σημαντικές λέξεις κλειδιά.
9. **Keywords:** Οι λέξεις κλειδιά που συνδέονται με το άρθρο , το οποίο είναι εξαιρετικά σημαντικό καθώς περιλαμβάνει τα βασικά θέματα του άρθρου και μπορεί να χρησιμοποιηθεί για να εντοπιστούν πιο ακριβή αποτελέσματα
10. **Second Headline:** Δευτερεύον τίτλος , μπορεί να χρησιμοποιηθεί κατά την αναζήτησή καθώς περιέχει επιπλέον πληροφορίες για το άρθρο.

Το author , category , section και second headline αρχικά μου φαίνονται λιγότερο σημαντικά ωστόσο θα μπορούσε να χρησιμοποιηθούν.

Τα υπόλοιπα πεδία όπως index και url δεν έχει νόημα να χρησιμοποιηθούν , όσο για το date published , θα μπορούσα να φτιάξω μια μηχανή αναζήτησης με βάση την ημερομηνία σε java αλλά νομίζω είναι εκτός πλαισίων της άσκησης , ζητείται αναζήτηση με λέξεις κλειδιά

Περιγραφή σχεδιασμού

Εισαγωγή: Στόχος είναι η κατασκευή μιας αποτελεσματικής μηχανής αναζήτησης για ειδησεογραφικά άρθρα αξιοποιώντας την βιβλιοθήκη Lucene. Το σύστημα θα υποστηρίζει:

- Αναζήτηση με λέξεις-κλειδιά σε συγκεκριμένα πεδία (π.χ., τίτλος, συγγραφέας, κείμενο άρθρου),
- Ιστορικό προηγούμενων ερωτημάτων,
- Διανυσματική αναζήτηση με χρήση προεκπαιδευμένων embeddings,
- Ευχάριστη παρουσίαση των αποτελεσμάτων με δυνατότητα φιλτραρίσματος και ταξινόμησης.

Ανάλυση Κειμένου και Κατασκευή Ευρετηρίου: Κατά την προετοιμασία των άρθρων για εισαγωγή στο ευρετήριο , θα εφαρμοστούν:

- **Stemming** (π.χ. με PorterStemFilter),
- **Απαλοιφή stop words** (με χρήση StopFilter και custom stopword set),
- **Lowercasing** (με LowerCaseFilter),
- **Επέκταση συνωνύμων** (π.χ. με SynonymGraphFilter, για να το κάνω αυτό βρήκα ένα αρχείο WordnetSynonyms.csv και με ένα python προγραμμα βρήκα μόνο τα συνώνυμα για τις λέξεις που αναφέρονται στα άρθρα και το αποθηκεύω σε ένα άρθρο FilteredSynonyms.csv, θα παραθέσω τον κώδικα της python αλλά όχι το WordnetSynonyms.csv καθώς είναι πολύ μεγάλο αρχείο). Δουλεύει απλά και τα συνώνυμα σε αυτό το αρχείο δεν είναι παντα ικανοποιητικά.

Οι παραπάνω ενέργειες θα υλοποιηθούν με custom analyzer για κάθε πεδίο όπου χρειάζεται , καθώς το κάθε πεδίο μπορεί να χρειάζεται διαφορετική ανάλυση , π.χ. στον author πολύ πιθανόν να μην χρειάζεται stemming για να μην χαλάσει το όνομα.

Μονάδα Εγγράφου

Η βασική μονάδα είναι το άρθρο. Κάθε άρθρο θα καταχωρείται στο ευρετήριο ως ένα Document.

Πεδία (Fields)

Τα πεδία που θα χρησιμοποιηθούν στο Document είναι:

- **Headline ,second Headline** – Τίτλος άρθρου (TextField, με προεπεξεργασία),
- **Description** - Περιγραφή (TextField, με προεπεξεργασία),
- **Text** – Πλήρες άρθρο (TextField, με προεπεξεργασία),
- **Keywords** – Λέξεις-κλειδιά (TextField, με προεπεξεργασία),
- **author** – Συγγραφέας (TextField, με προεπεξεργασία),
- **Category, section**— Κατηγορία άρθρου (TextField, με προεπεξεργασία),
- **Date** – Ημερομηνία δημοσίευσης (TextField date-month-year)
- Για αλφαβητική υποστήριξη: **Headline(SortedDocValuesField)**
- Για διανυσματική αναζήτησή :

- `embedding` – Διανυσματική αναπαράσταση (`KnnVectorField`).

Αναζήτηση: Η αναζήτησή θα υποστηρίζει δύο βασικούς τύπους:

1. **Αναζήτησή με λέξεις-κλειδιά:** Θα γίνεται μέσω το `QueryParse` ή `MultiFieldQueryParser` για τα κείμενα , με δυνατότητα αναζήτησης σε πολλά πεδία ταυτόχρονα

π.χ.

- Πεδίο: `headline:text` ή `author:John`
- API: `QueryParser.parse()`, `IndexSearcher.search()`

2. **Αναζήτησή με διανύσματα:**

π.χ.

- Υπολογίζεται `embedding` του ερωτήματος μέσω DJL.
- Χρήση `KnnVectoryQuery` για εντοπισμό των πιο κοντινών άρθρων.
- API: `KnnVectoryQuery`, `IndexSearcher.search()`

Ιστορικό Ερωτημάτων:

Τα τελευταία 20 ερωτήματα και τα 5 πρώτα άρθρα από κάθε αναζήτηση θα αποθηκεύονται τοπικά (σε αρχείο ή in-memory δομή) για:

- Βελτίωση μελλοντικών αναζητήσεων (π.χ. re-ranking),
- Ανάλυση προτιμήσεων χρήστη, ενισχύω τα άρθρα που αποτελούσαν αποτελέσματα παλιών ερωτήσεων

Παρουσίαση Αποτελεσμάτων:

Τα αποτελέσματα θα παρουσιάζονται:

1. Ως ένα webpage με το τίτλο , το description και το υλικό του field που διάλεξε
2. Τα αποτελέσματα θα εμφανίζονται σε σελίδες των 10 άρθρων η κάθε μια
3. Με τονισμένες λέξεις-κλειδιά (highlighting) στο συγκεκριμένο field που διάλεξε.

- API π.χ.: Highlighter, QueryScore

3. Δυνατότητα ταξινόμησης με βάση τον τίτλο:

- Αλφαβητικά χρησιμοποιώντας Sort και SortField
- 4. Για κάθε άρθρο θα υπάρχει επιλογή εμφάνισης ολόκληρου του άρθρου με όλα τα στοιχεία

5. Το UI θα υλοποιηθεί με springboot αφού τρέξετε το πρόγραμμα απλά ανοίγουμε ένα browser στο localhost:8080

Μελλοντικές σκέψεις και βήματα...(δεν είμαι σίγουρος ακόμα)

- Επιλογή/εκπαίδευση μοντέλου για embeddings (π.χ., all-MiniLM-L6-v2 μέσω DJL).
- Δημιουργία custom analyzers.
- Σχεδιασμός του UI.
- Υλοποίηση του Search History module.